



Day 8: Building Dynamic Web Pages with PHP

From Simple Forms to Dynamic Applications

FULL STACK BOOTCAMP

DAY 8 OF 13

2 HRS 30 MIN



Yesterday We Collected Data. But Can We Make It Look Like This?

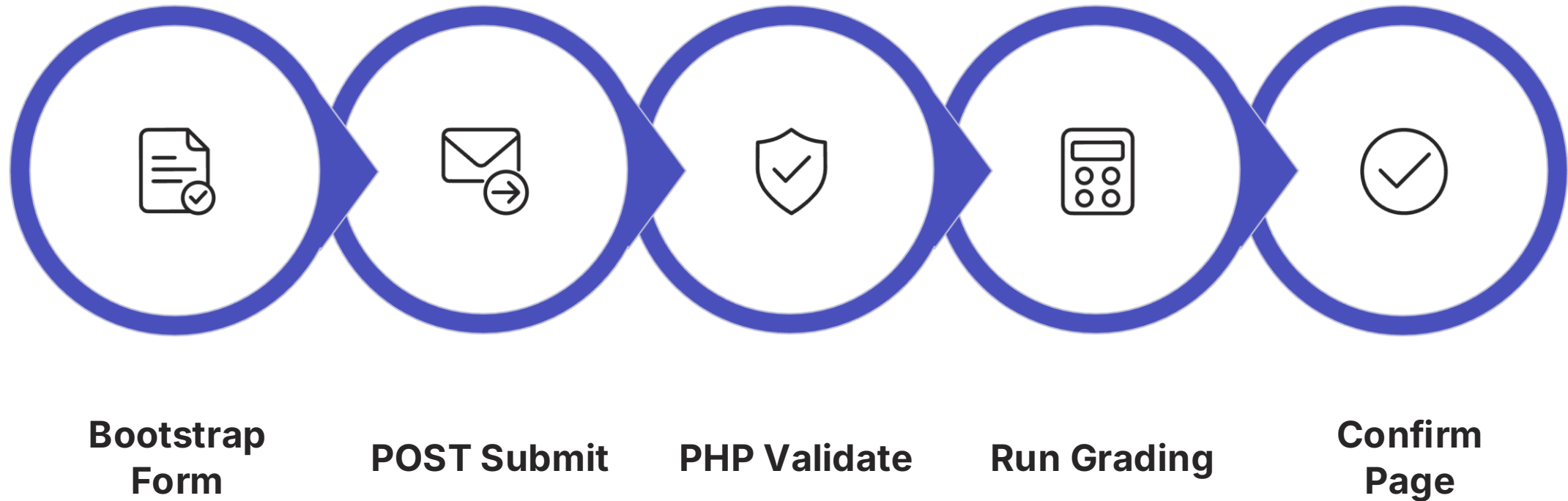
You built a form. A user filled it in. And then... nothing special happened. Today we change that. Think about Google Forms — when you submit, you land on a polished, personalized Thank You page. How is that page generated? Hint: it's not magic. It's PHP reading what you submitted and building the page dynamically, in real time.



Pause. Let students think. Don't answer immediately — let the question land.

Today's Mission

By end of session, every team ships a **Student Registration Confirmation System** — a real, working multi-page PHP application.



This is not a toy project. It uses Bootstrap, PHP, validation, functions, and reusable includes — everything a real app needs. Keep this flow in mind throughout the day.

⚡ Rapid-Fire Recap — 5 Minutes

Quick pair discussion before we push forward. No notes, no Googling — just talk it out with your partner.

How do you run a PHP file?

Via `localhost` in XAMPP — never directly from the filesystem.

What is `localhost`?

Your own machine acting as a web server, served by Apache inside XAMPP.

What does `echo` do?

Outputs text or HTML directly into the browser response.

HTML vs PHP?

HTML is static markup; PHP runs server-side and generates HTML dynamically.

What is `$_POST`?

A superglobal array holding form data sent via the POST method.

GET vs POST

GET — Data in the URL

```
<form method="get" action="result.php">
  <input name="name" />
  <button>Submit</button>
</form>
```

URL becomes: `result.php?name=Martin`

Use for search queries, filters, bookmarkable pages.

POST — Data hidden in the body

```
<form method="post" action="result.php">
  <input name="name" />
  <button>Submit</button>
</form>
```

URL stays clean: `result.php`

Use for login, registration, sensitive or large data.

 Live Code Now: Create both forms side by side. Open browser and compare URLs after each submit. See the difference visually.

Mission: Feedback Form via POST

Your Mission

Build a Bootstrap feedback form with three fields:

- **Name** — text input
- **Email** — email input
- **Message** — textarea

On submit, display the submitted data using `$_POST` in a styled result page.

Code Challenge

Upgrade your result page to display a personalized greeting using the submitted name:

```
<?php
$name = $_POST['name'];
echo "<h2>Thank you,
    $name!</h2>";
?>
```

Style the output using a Bootstrap `alert-success` card. Make it look polished.



MODULE 2

Conditional Logic in PHP

Conditions let PHP make decisions. Syntax is identical to JavaScript — but remember the semicolons and PHP tags.

```
<?php
$cgpa = $_POST['cgpa'];

if ($cgpa >= 9) {
    echo "
```

Mission: Student Grade Checker

Your Mission

Build a form where a student enters their **CGPA**. On submit, PHP reads the value and outputs a personalized performance message.

- Input: one number field for CGPA (e.g. 8.7)
- Output: performance message based on range
- Wrap everything in a Bootstrap card for polish

Code Challenge

Level up the output by using different **Bootstrap alert colors** to reflect each grade band:

- `alert-success` → Excellent
- `alert-primary` → Very Good
- `alert-warning` → Good
- `alert-danger` → Keep Improving

The color should change *dynamically* based on the PHP condition result.



MODULE 3

PHP Functions — Write Once, Reuse Everywhere

Functions eliminate repetition. Define once, call anywhere. PHP functions work just like JavaScript functions — same concept, slightly different syntax.

```
<?php
function greetStudent($name) {
    return "Welcome, $name!"
}
```

Mission: Welcome Card Generator

Your Mission

Build a form that collects three inputs:

- **Student Name**
- **Branch** (e.g. Computer Science)
- **College Name**

On submit, call a PHP function that returns a formatted, Bootstrap-styled **Welcome Card** displaying all three fields.

Code Challenge

Add the current date to your welcome card using PHP's built-in date function:

```
<?php
function getFormattedDate() {
    return date("l, F j, Y");
}
// Output: Monday, June 9, 2025
?>
```

Display it below the student's name on the card. Bonus: add a greeting that changes by time of day using `date('H')`.

Reusable Pages with `include`

Professional PHP apps never repeat code. A single `header.php` powers every page of the site — change it once, update everywhere.

File Structure

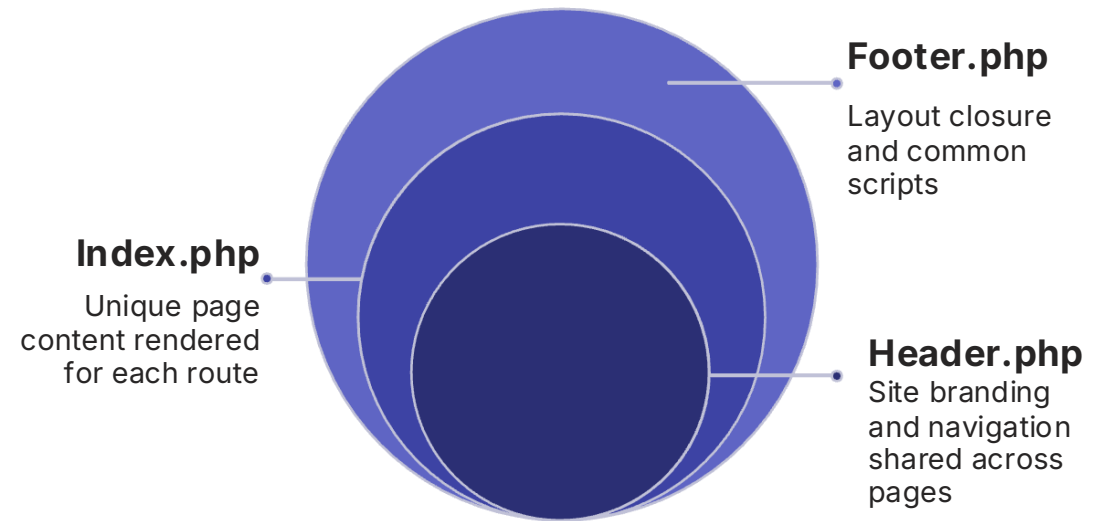
```
/project
├── header.php
├── footer.php
└── index.php
```

`index.php`

```
<?php include 'header.php'; ?>

<main class="container">
  <h1>Welcome!</h1>
  <p>Page content here.</p>
</main>

<?php include 'footer.php'; ?>
```



This pattern is used by every major PHP framework: Laravel, WordPress, CodeIgniter. Master it now and it applies everywhere.

Mission: Split Your Site into Reusable Files

Your Mission

Refactor your existing project into three separate files:

- **header.php** — Bootstrap navbar, meta tags, CSS links
- **footer.php** — copyright, scripts, closing tags
- **index.php** — unique page content only

Use `include` to assemble the full page. Confirm it renders identically in the browser via `localhost`.

Code Challenge

Add a Bootstrap navbar to `header.php` with links to at least three pages. Then **highlight the active page** dynamically using a PHP variable:

```
<?php $page = "home"; ?>
<?php include 'header.php'; ?>
```

Inside `header.php`, add `class="active"` to the correct nav link using an `if` condition on `$page`.

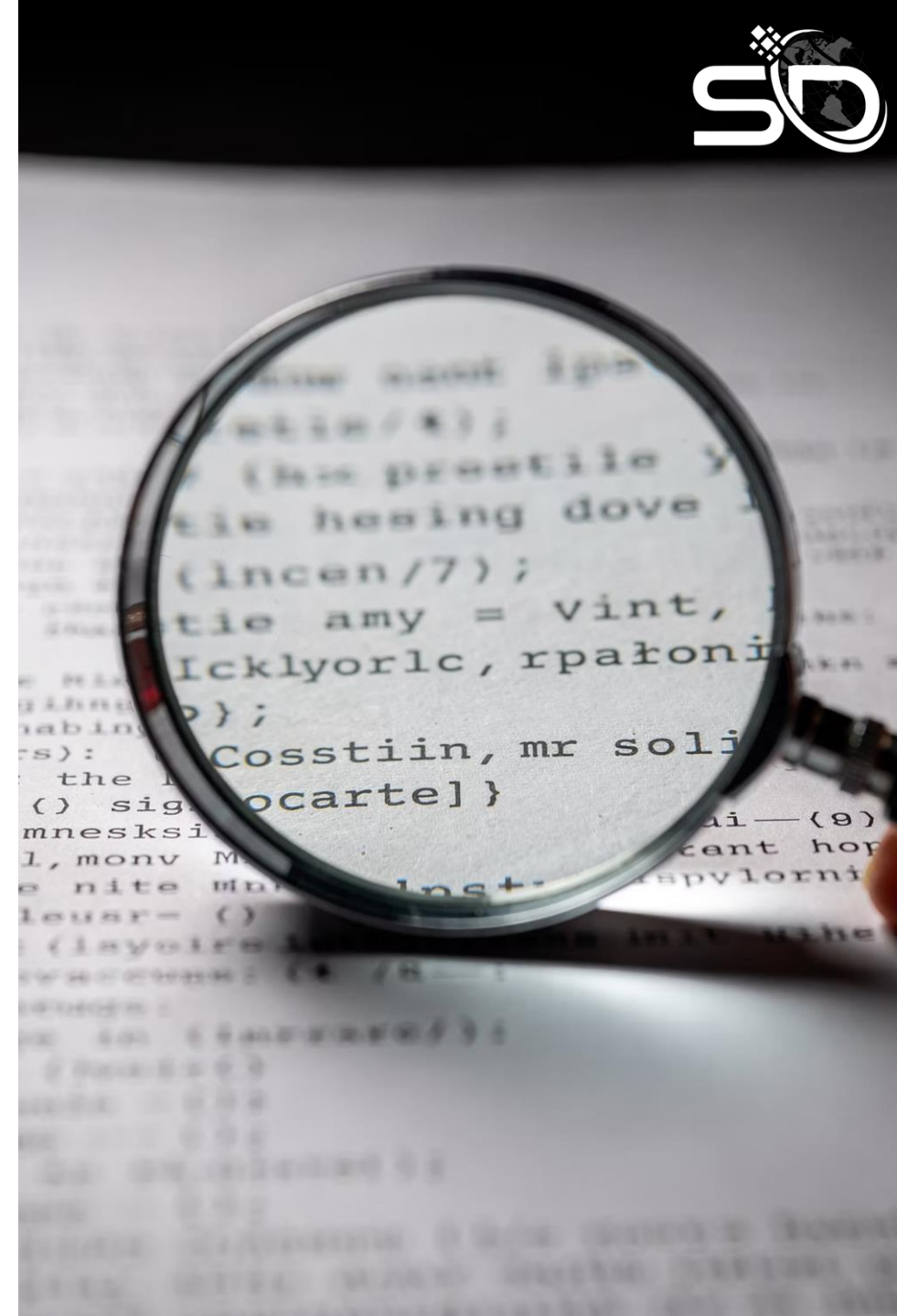
Spot the Bug — Can You Find All 5?

The following PHP page is broken. Working in pairs, identify every bug before the reveal. There are exactly **5 bugs** hidden in this code.

```
<?php
$name = $_POST['student_name']
$cgpa = $_GET['cgpa'];

if ($cgpa >= 9) {
    $grade = "Excellent"
} else {
    $grade = "Keep Improving";
}

echo "<h2>Welcome $Nam!</h2>";
?>
<?php include 'hedr.php'; ?>
```



Build Sprint: Student Registration Confirmation System

Time to put it all together. This is a hackathon-style sprint — build fast, build clean, build something you're proud to demo.

✓ Bootstrap Form

Name, Email, CGPA, Branch, College — styled with Bootstrap 5 form components.

✓ POST + Validation

All fields sent via POST. PHP checks for empty inputs before processing.

✓ Grade Logic

Use your `calculateGrade()` function with Bootstrap alert colors.

✓ Welcome Card

A polished confirmation card showing all submitted details and today's date.

✓ Header & Footer

Use `include` for shared layout — navbar, branding, footer.

★ Bonus

Profile photo placeholder, Font Awesome icons, gradient background on confirmation card.

5 PHP Interview Questions

These questions come up in real internship and junior developer interviews. Answer without looking at your code.

Q1

What is the difference between `GET` and `POST`? When would you use each?

Q2

Why would you use `include()` instead of copying code into every page?

Q3

What is `$_POST`? What data type is it?

Q4

What does this output?

```
$x=7; if($x>8){echo "A";}else{echo "B";}
```

Q5

Why use a function instead of writing the same logic multiple times in a page?

Improve Your Project Tonight

Your build sprint was 30 minutes. Tonight you have hours. Elevate your Registration Confirmation System into something portfolio-worthy.>

Feature Upgrades

→ Student Photo Upload UI

Add a file input field styled with Bootstrap. (Display placeholder for now — no server upload needed.)

→ More Fields

Add Gender (radio), Course (dropdown), and Address (textarea) to the form.

→ Better Bootstrap Design

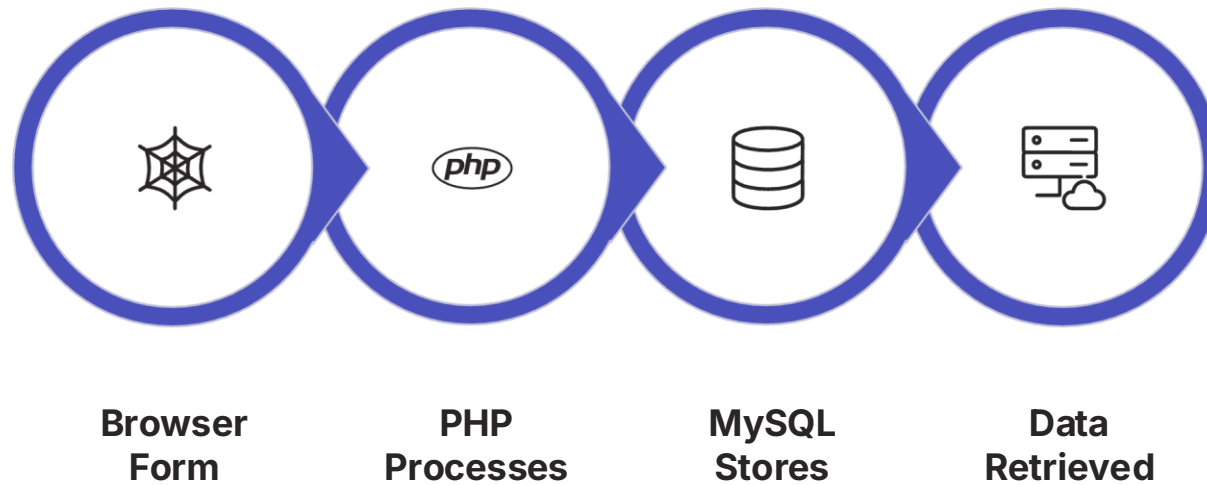
Use cards, badges, icons (Font Awesome), and a gradient header on the confirmation page.

Submit to GitHub

Push your completed project to your GitHub repository with a clear `README.md` that describes what the project does.

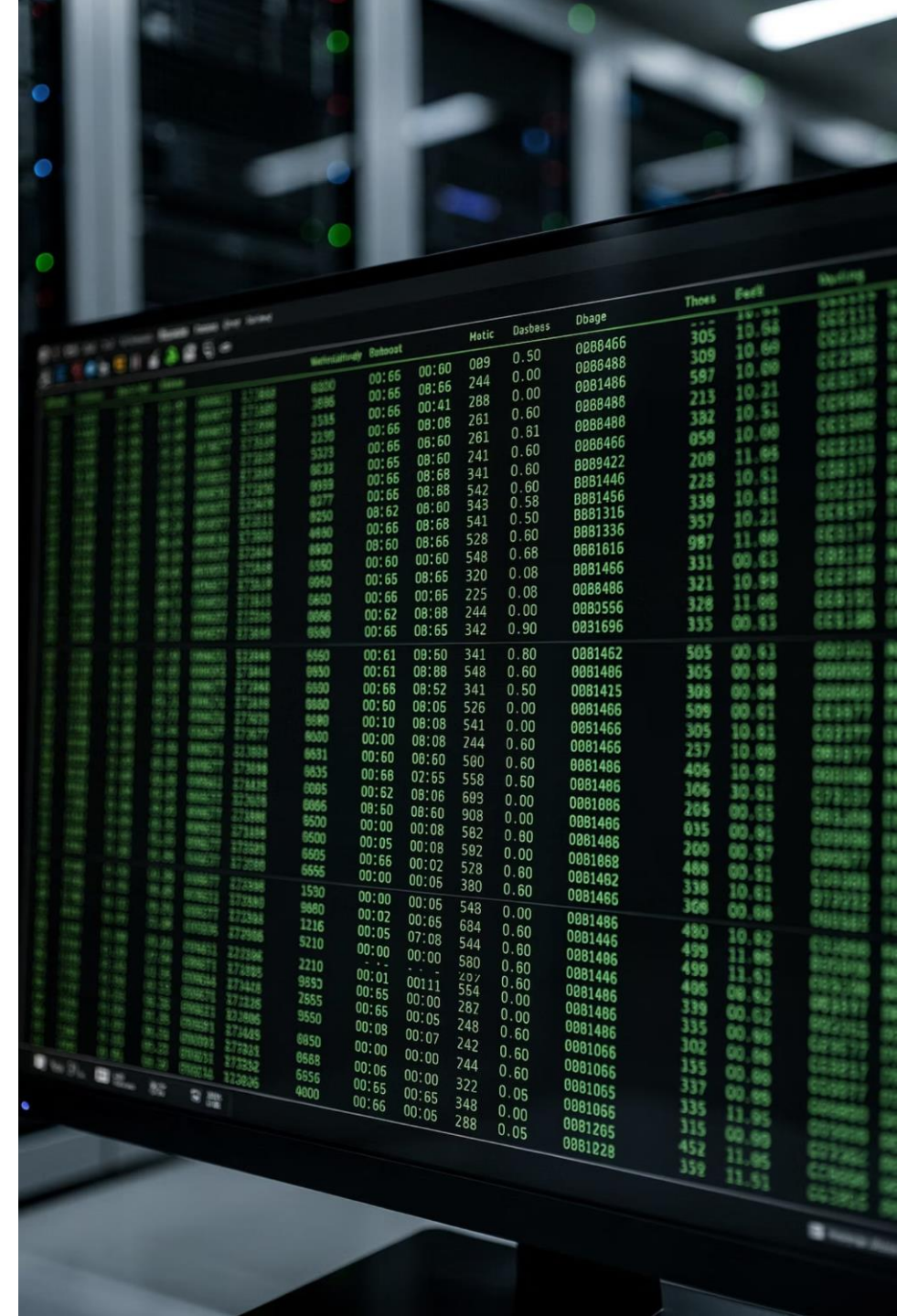
- ✅ Students who complete the bonus features and push to GitHub on time get priority demo slots on Day 9.

Tomorrow: Saving Data Permanently



Today your application processed information. Tomorrow you'll store it permanently inside a database — and retrieve it whenever you want.

Day 9 introduces **MySQL**, **phpMyAdmin**, and your first `INSERT` query. Everything you built today becomes the front-end of a real database-driven application.



See You Tomorrow

Push your project. Document your code. Get some rest — Day 9 is where things get seriously powerful.



Push to GitHub

Commit your Day 8 project with a clear README before midnight.



Review Today's Code

Read through your functions and includes one more time — reinforce the patterns.



Preview MySQL

Open phpMyAdmin at `localhost/phpmyadmin` — we start there tomorrow.

FULL STACK BOOTCAMP — DAY 8 COMPLETE